Notes on: Communication Protocols_from_0

1.) Messaging Protocols (MQTT, CoAP, XMPP)

Introduction to IoT Messaging Protocols

• Messaging protocols are rules for how IoT devices exchange data; they define the format and sequence of messages.
• They work at the Application Layer of the network stack; on top of transport protocols like TCP or UDP.
• Think of them as the language devices speak; while TCP/IP is the postal service that carries the letters.
• In IoT, these protocols must be efficient; handling many devices with limited power and bandwidth.
• We will discuss three important ones: MQTT, CoAP, and XMPP.

1. MQTT - Message Queuing Telemetry Transport

1.1 Core Concept
• MQTT is a lightweight messaging protocol; designed for M2M (Machine-to-Machine) communication.
• It uses a Publish-Subscribe (Pub/Sub) communication model.
• It is not a direct device-to-device communication.
• Analogy: A YouTube channel; the creator (Publisher) uploads a video; subscribers get notified. The creator doesn't send the video to each subscriber individually; YouTube (the Broker) handles it.

1.2 Key Components
• Publisher: The client that sends messages; e.g., a temperature sensor.
• Subscriber: The client that receives messages; e.g., a mobile app or a server application.
• Broker: The central server that receives all messages from publishers; it then forwards them to the right subscribers. The broker is the heart of MQTT.
• Topic: A string that acts as a message filter or channel; e.g., **home/livingroom/temperature**. Messages are published to topics.

1.3 How MQTT Works
• A sensor (Publisher) measures temperature.
• It sends the data (e.g., **25 C**) to the MQTT Broker on the topic **home/livingroom/temperature**.
• A mobile app (Subscriber) has already subscribed to the topic **home/livingroom/temperature** with the Broker.
• The Broker sees the new message on that topic; and forwards it to the mobile app.
• The Publisher and Subscriber are decoupled; they never know about each other; they only know the Broker.

1.4 Key Features
• Lightweight: Small message headers; minimal overhead; good for low bandwidth networks.
• Runs over TCP: Ensures reliable, ordered message delivery.
• Quality of Service (QoS): Defines the guarantee of message delivery.
• - QoS 0 (At most once): Fire and forget; no delivery confirmation; fastest but least reliable.
• - QoS 1 (At least once): Message is guaranteed to be delivered; but might arrive multiple times. Requires acknowledgement (ACK).
• - QoS 2 (Exactly once): Message is guaranteed to be delivered exactly once; most reliable but slowest due to a four-part handshake.
• Last Will and Testament (LWT): A pre-defined message the broker sends on behalf of a client if it disconnects ungracefully (e.g., power loss).
• Retained Messages: The broker can store the last message on a topic; new subscribers to that topic immediately receive this last known value.

1.5 When to Use MQTT
- For one-to-many or many-to-many communication.
- When network is unreliable and message delivery is critical.
- When devices have limited bandwidth.
- Common use cases: Smart home automation; industrial IoT (IIoT) monitoring; fleet management.

2. CoAP - Constrained Application Protocol

2.1 Core Concept
- CoAP is designed for constrained devices and networks (e.g., 6LoWPAN).
- It follows a Request-Response communication model; similar to HTTP.
- Analogy: Browsing a website. You send a request for a webpage (e.g., GET google.com); the server sends a response with the page content.

2.2 Key Design
- It's a web protocol for IoT; often called the **HTTP for constrained devices**.
- Communication is one-to-one; between a client and a server.
- Uses URIs (Uniform Resource Identifiers) to identify resources on a server; e.g., **coap://[device_ip]/sensors/temperature**.

2.3 How CoAP Works
- A client (e.g., a computer) wants to know the temperature from a sensor (Server).
- Client sends a GET request to the sensor's URI.
- The sensor server processes the request; and sends back a response with the temperature value.
- It uses methods like GET, POST, PUT, DELETE; just like HTTP.

2.4 Key Features
- Runs over UDP: User Datagram Protocol; which is connectionless and faster than TCP; but less reliable by itself.
- Built-in Reliability: CoAP adds a simple reliability layer on top of UDP.
- - Confirmable (CON) Messages: Require an acknowledgement (ACK). If no ACK is received, the message is re-sent. This is like QoS 1 in MQTT.
- - Non-confirmable (NON) Messages: Fire and forget; no ACK is needed. This is like QoS 0.
- Low Overhead: Has a very small, binary header.
- Resource Discovery: A client can ask a server to list all its available resources (URIs).
- Observation: A client can **observe** a resource. The server will then automatically send updates to the client whenever the resource's state changes. This provides a pub/sub-like feature.

2.5 When to Use CoAP
- For communication with very low-power, battery-operated devices.
- In low-bandwidth, lossy networks like 6LoWPAN.
- For simple state transfer and request/response scenarios.
- Common use cases: Smart lighting; environmental sensing; building automation.

3. XMPP - Extensible Messaging and Presence Protocol

3.1 Core Concept
- XMPP is a protocol for real-time communication; originally built for instant messaging (IM).
- Its core strengths are addressing, presence, and security.
- Analogy: A chat application like WhatsApp or Google Chat. Each user has a unique address; you can see if they are online or offline (presence).

3.2 Key Design
- It is decentralized and federated; meaning anyone can run their own server; and these servers can talk to each other.
- It uses XML (Extensible Markup Language) to structure data in **stanzas**.
- Each device or user is identified by a Jabber ID (JID); which looks like an email address (e.g., sensor1@myhome.com).

3.3 How XMPP Works
- An IoT device (Client) with a JID connects to its XMPP Server.
- It can send a message (an XML stanza) to another JID; for example, myphone@myhome.com.
- The server routes the message to the destination client or another server.
- It supports **presence** information; the device can broadcast its status (e.g., online, offline, busy).

3.4 Key Features
- Extensible: Can be extended with new capabilities via XMPP Extension Protocols (XEPs).
- Addressing: The JID provides a unique, human-readable address for every device.
- Presence: Built-in feature to know the status of devices; which is very useful in IoT.
- Security: Mature and robust security features are integrated (TLS, SASL).
- Verbose: Using XML makes messages large and text-heavy; this can be a problem for constrained devices. This is its main disadvantage in IoT.

3.5 When to Use XMPP in IoT
- When a unique, routable address for each device is important.
- When human-to-device communication is a key feature (e.g., **chatting** with your home appliances).
- When presence information is crucial for the application logic.
- Less common for simple sensor data telemetry due to its overhead compared to MQTT or CoAP.

4. Quick Comparison and Summary

- Communication Model:
- - MQTT: Publish/Subscribe (many-to-many via broker).
- - CoAP: Request/Response (one-to-one).
- - XMPP: Messaging & Presence (peer-to-peer, can be federated).

- Based On:
- - MQTT: TCP (Reliable, connection-oriented).
- - CoAP: UDP (Fast, connectionless).
- - XMPP: TCP (Reliable, connection-oriented).

- Message Format & Overhead:
- - MQTT: Binary, very low overhead.
- - CoAP: Binary, very low overhead.
- - XMPP: XML, high overhead (verbose).

- Addressing:
- - MQTT: Topic strings (e.g., **sensor/data**).
- - CoAP: URI paths (e.g., **/sensors/temp**).
- - XMPP: Jabber ID (JID) (e.g., **device@domain.com**).

- Best For:
- - MQTT: Reliable telemetry data from many devices to a central point.
- - CoAP: Simple state transfer with battery-powered devices in lossy networks.
- - XMPP: Applications needing strong identity, presence, and federation.

- Final Thought:
- The choice of protocol is not about which is **best**.
- It depends entirely on the requirements of your IoT application.
- For lightweight data collection, MQTT is often the standard.
- For web-like interaction with constrained devices, CoAP is the choice.
- For complex systems with addressing and presence needs, XMPP might be considered.

2.) Transport Protocols (Introduction of BLE, Introduction to Li-Fi)

Transport Protocols: Introduction to BLE and Li-Fi

1- Role of Lower-Level Protocols in IoT

- You have learned about Messaging Protocols; like MQTT and CoAP.
- These are Application Layer protocols; they define the message format.
- They decide WHAT data is sent and HOW it is structured.
- But they do not handle the physical transmission of data.
- For data to travel, we need lower-level protocols.
- These protocols work at the Physical (PHY) and Data Link Layers.
- They define HOW the data is actually sent over a medium; like air or light.
- Today, we will learn about two important technologies for IoT at this level.
- 1. Bluetooth Low Energy (BLE) - using radio waves.
- 2. Li-Fi - using light waves.

- ---

Bluetooth Low Energy (BLE)

- ---

2- Introduction to BLE

- BLE stands for Bluetooth Low Energy.
- It is a wireless personal area network (WPAN) technology.
- Designed for short-range communication.
- Main goal: very low power consumption.
- It is different from Classic Bluetooth.
- Classic Bluetooth is for continuous data streaming; like audio headphones.
- BLE is for short, periodic bursts of data; like a temperature sensor sending a reading every minute.
- BLE operates in the 2.4 GHz ISM (Industrial, Scientific, and Medical) band.
- This is the same frequency band as Wi-Fi and Classic Bluetooth.

3- BLE Key Concepts and Architecture

- Devices in BLE have specific roles.
- Central Role: This device scans for other devices; it initiates connections.
- Example: Your smartphone or a computer.
- Peripheral Role: This device advertises its presence; it waits for a connection.
- Example: A fitness band, a smartwatch, or an IoT sensor.
- A central device can connect to multiple peripheral devices.
- A peripheral can only be connected to one central device at a time.

4- GATT: The Language of BLE

- Once connected, devices use GATT to communicate.
- GATT stands for Generic Attribute Profile.
- It defines the structure of data and how it is exchanged.
- Think of it as a small, simple database on the peripheral device.
- GATT has a hierarchical structure:
- - Profile: A collection of services needed for a specific application.
- - Service: A collection of related data points, called characteristics.
- Example: A **Heart Rate Service**.
- - Characteristic: The actual data value.
- Example: A **Heart Rate Measurement** characteristic which holds the BPM value.
- A characteristic has a value; properties (read, write, notify); and descriptors (metadata).
- Each service and characteristic has a unique ID called a UUID.
- UUID: Universally Unique Identifier; a 128-bit number.

5- How BLE Devices Discover Each Other

- Discovery process involves two main actions:

- 1. Advertising: The Peripheral device sends out small packets of data.
- These are called advertising packets.
- They are broadcast on 3 specific **advertising channels**.
- This packet says **I am here, and this is my name/service**.
- 2. Scanning: The Central device listens on these advertising channels.
- When it finds a device it wants, it can request to establish a connection.
- This process is very power-efficient for the peripheral device.

6- Simplified BLE Protocol Stack

- The BLE protocol is a stack of layers; each with a specific job.
- - Physical Layer (PHY):
- Controls the radio transmitter and receiver.
- Manages the 2.4 GHz frequency band.
- - Link Layer (LL):
- Manages the connection state (advertising, scanning, connected).
- Defines device roles (Master/Central, Slave/Peripheral).
- Handles packet formatting and error checking.
- - Host Controller Interface (HCI):
- A standard interface layer.
- Allows the main processor (Host) to talk to the radio chip (Controller).
- - Generic Access Profile (GAP):
- Controls discovery and connection.
- Manages advertising and scanning.
- Defines how devices appear to the outside world.
- - Generic Attribute Profile (GATT):
- Controls data exchange after a connection is made.
- Manages the services and characteristics data structure.
- You interact with GATT when you build a BLE application.

7- Advantages of BLE for IoT

- Ultra-Low Power: Devices can run for months or even years on a single coin cell battery.
- Low Cost: BLE chips are inexpensive; easy to add to any device.
- Widespread Availability: Almost every smartphone, tablet, and laptop has BLE built-in.
- Ease of Use: Simple to set up connections.
- Good Range: Typically 10 to 100 meters, depending on the environment.

8- Common BLE Applications in IoT

- Wearable Devices: Fitness trackers, smartwatches.
- Healthcare: Glucose monitors, pulse oximeters, smart thermometers.
- Smart Home: Smart locks, light bulbs, temperature sensors.
- Beacons: For indoor navigation and proximity marketing in malls or museums.
- Asset Tracking: To monitor the location of equipment in a factory or hospital.

- ---

Li-Fi (Light Fidelity)

- ---

9- Introduction to Li-Fi

- Li-Fi stands for Light Fidelity.
- It is a wireless communication technology.
- It uses light waves instead of radio waves to transmit data.
- It uses the visible light spectrum, but can also use infrared or ultraviolet.
- The core idea is to modulate the intensity of an LED light bulb.
- The LED is turned on and off extremely quickly; faster than the human eye can see.
- A '1' can be represented by 'LED ON' and a '0' by 'LED OFF'.

• A receiver detects these changes and decodes them back into data.
• Analogy: A super-fast version of Morse code using a flashlight.

10- Key Components of a Li-Fi System

• A Li-Fi setup is quite simple in concept.
• 1. LED Light Source (Transmitter):
• A standard LED bulb is fitted with a signal processing chip.
• This chip modulates the light according to the input data stream.
• 2. Photodetector (Receiver):
• A light sensor (like a photodiode) is used to receive the light signals.
• It converts the changes in light intensity back into an electrical signal.
• The signal is then decoded into data.

11- Li-Fi vs. Wi-Fi: A Comparison

• Medium:
• - Li-Fi: Uses light waves (Visible Light Communication - VLC).
• - Wi-Fi: Uses radio waves (Radio Frequency - RF).
• Speed:
• - Li-Fi: Potentially much faster; can achieve speeds in Gbps.
• - Wi-Fi: Typically offers speeds in Mbps.
• Spectrum:
• - Li-Fi: Uses the light spectrum; it is 10,000 times larger than the radio spectrum. It is free and unregulated.
• - Wi-Fi: Uses the radio spectrum; it is licensed, crowded, and limited.
• Security:
• - Li-Fi: Highly secure; light cannot pass through walls. Data is confined to the room.
• - Wi-Fi: Less secure; radio waves can pass through walls and be intercepted from outside.
• Interference:
• - Li-Fi: Immune to RF interference from devices like microwaves or cordless phones.
• - Wi-Fi: Susceptible to RF interference.
• Range & Coverage:
• - Li-Fi: Shorter range and requires a direct line-of-sight.
• - Wi-Fi: Longer range and can penetrate through walls and obstacles.

12- Advantages of Li-Fi for IoT

• High Speed: Excellent for high-bandwidth IoT applications like video streaming from security cameras.
• Enhanced Security: Perfect for environments that require high data security; like banks, military, and corporate offices.
• No RF Interference: Can be used in RF-sensitive areas; like hospitals (around MRI machines), airplanes, and industrial plants.
• High Density: Each light source in a room can be a separate access point; this prevents network congestion in crowded areas.
• Dual Use: The same infrastructure for lighting can also be used for data communication.

13- Disadvantages and Challenges of Li-Fi

• Line-of-Sight: The receiver must be able to **see** the LED transmitter. Any obstacle will block the signal.
• Sunlight Interference: Bright sunlight can interfere with the photodetector, reducing performance.
• Uplink Problem: Sending data back from the device to the light source (uplink) is difficult. It often requires a separate technology, like an infrared transmitter on the device.
• Infrastructure Cost: Requires installing new Li-Fi enabled LED bulbs.

14- Common Li-Fi Applications in IoT

• Smart Buildings/Offices: Providing secure and fast internet access through ceiling lights.
• Hospitals: Transmitting patient data safely without interfering with medical equipment.
• Underwater Communication: Radio waves don't travel well underwater; light does. Useful for underwater drones and sensors.
• Vehicle-to-Vehicle (V2V) Communication: Using car headlights and taillights to exchange data for safety and traffic management.
• Airline Cabins: Providing internet to passengers without RF interference.

15- Summary and Conclusion

• BLE and Li-Fi are physical/link layer technologies; they move the bits.
• Protocols like MQTT and CoAP run on top of them; they structure the message.
• BLE is the best choice for:
• - Low-power, battery-operated devices.
• - Low data rate applications.
• - Short-range wireless connectivity.
• Li-Fi is the best choice for:
• - High-speed data transmission.
• - Highly secure environments.
• - Areas where RF is prohibited or congested.
• The arrangement of these devices in a network; for example, in a star or mesh pattern; is defined by network topologies, a topic you will explore next.

3.) Basics of Sensor Network Topologies (Point to Point Topology, Mesh topology, Ring topology, Star Topology)

Introduction to Sensor Network Topologies

• In the world of IoT, devices need to talk to each other; this is communication.
• We already know about messaging protocols like MQTT; they define the language or format of messages.
• We also know about transport methods like BLE; they define the medium for communication.
• Now, we study Network Topology; it defines the structure or layout of the network.
• Topology is the 'map' of how sensor nodes, gateways, and devices are connected.
• It answers the question: **How are the devices arranged and linked?**.
• The choice of topology affects everything; cost, reliability, power usage, and how easy it is to expand the network.
• There are two views of topology;
• 1. Physical Topology; the actual physical layout of devices and cables.
• 2. Logical Topology; the path data takes to travel between devices.

• --
1. Point-to-Point (P2P) Topology

• Definition: A direct, dedicated link connecting exactly two devices or nodes.
• It is the simplest form of network connection.
• Think of it as a private conversation between two people.

• How it Works:
• Node A is connected only to Node B.
• Data flows directly from sender to receiver.
• No other device can access this link.
• Communication is direct; without any intermediate device or hub.

• Analogy:
• A single telephone line connecting two houses.

• A TV remote controlling a specific TV.

• Key Characteristics:
• Security; Very high, as the link is private.
• Bandwidth; Fully dedicated, not shared.
• Simplicity; Very easy to set up and understand.

• Use Cases in IoT:
• A Bluetooth headset connected to a smartphone.
• A smart wearable (like a fitness band) sending data directly to a phone using BLE.
• Two microcontrollers communicating via a serial cable.

• Advantages:
• - High speed; data transfer is fast.
• - Highly secure and private.
• - Simple to troubleshoot; only two nodes and one link to check.

• Disadvantages:
• - Not scalable; cannot be used to connect many devices easily.
• - To connect 'n' devices, you would need n*(n-1)/2 links, which is very inefficient.

• --

2. Star Topology

• Definition: All devices (nodes) are connected to a single, central device.
• This central device is called a hub, switch, or gateway.
• The layout looks like a star or a wheel with spokes.

• How it Works:
• Peripheral nodes (sensors, actuators) are clients.
• The central hub is the server or master.
• To communicate, a node must send data to the central hub first.
• The hub then forwards the data to the destination node or an external network (like the internet).
• Nodes cannot communicate directly with each other.

• Analogy:
• A home Wi-Fi network; all your devices (laptop, phone, TV) connect to the Wi-Fi router.
• A manager (central hub) with their team members (nodes); all communication goes through the manager.

• Key Characteristics:
• Centralized Control; The hub manages and controls the entire network.
• Isolation; A failure in one node's link does not affect others.
• Easy Expansion; Adding a new node is simple; just connect it to the central hub.

• Use Cases in IoT:
• Home automation systems; smart lights, fans, and sensors connect to a central controller like Google Home or a Zigbee coordinator.
• Most Wi-Fi based IoT deployments.
• Corporate LAN networks.

• Advantages:
• - Easy to install and manage.
• - Adding or removing nodes is simple and does not disrupt the network.
• - Easy to find faults; if a node is down, only that link is affected.

• Disadvantages:
• - Single Point of Failure; If the central hub fails, the entire network stops working.
• - Performance bottleneck; Network performance depends on the power and capacity of the hub.

• - Can be costly; requires more cable than some other topologies and a powerful central device.

• --

3. Ring Topology

• Definition: Each node is connected to exactly two other nodes, one on each side.
• All nodes form a circular path or a closed loop.

• How it Works:
• Data travels around the ring from node to node.
• Usually, data flows in one direction (unidirectional).
• A special data packet called a **token** is often used (Token Ring).
• A node must wait for the token to arrive before it can send its own data.
• This prevents data collisions.

• Analogy:
• A game of 'passing the parcel' in a circle.
• A circular toy train track; the train (data) visits each station (node) in order.

• Key Characteristics:
• Orderly Communication; Every node gets an equal opportunity to transmit data.
• No Collisions; The token mechanism ensures only one node transmits at a time.
• Data Regeneration; Each node acts as a repeater, regenerating the signal before passing it on.

• Use Cases in IoT:
• Not very common in modern wireless IoT networks.
• Used in some high-performance industrial networks (e.g., FDDI).
• Sometimes used in backbone networks for telecom.

• Advantages:
• - Performs better than a star topology under heavy network traffic.
• - No central hub is needed to manage connectivity.
• - All nodes have equal access.

• Disadvantages:
• - Single Point of Failure; A failure in one cable or one node can break the entire ring.
• - Difficult to troubleshoot; A break could be anywhere in the ring.
• - Adding or removing nodes is difficult; it requires breaking the loop and disrupting the network.
• - Communication can be slow; data must pass through many intermediate nodes.

• --

4. Mesh Topology

• Definition: A network where devices are interconnected with many redundant connections.
• Every node is connected to several other nodes.
• It provides multiple paths for data to travel.

• Types of Mesh:
• - Full Mesh; Every node is directly connected to every other node. Very reliable but very expensive.
• - Partial Mesh; Some nodes are connected to all others, but some are connected only to nodes with which they exchange the most data. This is more practical.

• How it Works:
• Data can be sent from a source to a destination via different routes.
• The network is **self-healing**; if one path or node fails, the network automatically finds an alternative path.
• Nodes can act as simple endpoints and also as relays or routers for other nodes.

• Analogy:

- A spider's web; multiple interconnected threads.
- A city's road network; you can take many different routes to get from one point to another.

- Key Characteristics:
- Reliability; Extremely high, due to redundant paths.
- Robustness; The network can handle failures gracefully.
- Decentralized; No single point of failure.

- Use Cases in IoT:
- This is very important and widely used in IoT.
- Wireless Sensor Networks (WSNs) for smart agriculture, environmental monitoring.
- Smart city applications like smart street lighting.
- Protocols like Zigbee, Z-Wave, and Thread are built on mesh topology principles.

- Advantages:
- - Highly reliable and fault-tolerant.
- - Can handle high amounts of traffic.
- - Scalable; easy to add new nodes without disruption, which then expand the network's reach.

- Disadvantages:
- - Very complex to implement and manage.
- - High cost; requires a lot of connections (cabling in wired mesh, or complex routing in wireless).
- - Higher power consumption for nodes that act as routers, which is a key concern for battery-powered IoT devices.

- --

Quick Comparison Summary

- Point-to-Point:
- - Reliability: High (for that one link)
- - Scalability: Very Low
- - Cost: Low per link, high for many devices
- - Failure Impact: Only affects the two connected nodes.

- Star:
- - Reliability: Low (due to central hub dependency)
- - Scalability: High
- - Cost: Moderate
- - Failure Impact: Hub failure brings down the whole network; node failure affects only that node.

- Ring:
- - Reliability: Low
- - Scalability: Low
- - Cost: Low
- - Failure Impact: Single node/cable failure brings down the whole network.

- Mesh:
- - Reliability: Very High
- - Scalability: High
- - Cost: High
- - Failure Impact: Failure of one node or link has minimal impact; network reroutes data.

- --

Conclusion

- Choosing the right topology is a critical design decision in any IoT system.
- The choice depends on the application's specific needs.
- For a simple, two-device system; P2P is perfect.
- For a home network with a central controller; Star is ideal.

- For a large, mission-critical sensor field where reliability is key; Mesh is the best choice.
- In practice, many IoT networks use Hybrid Topologies, which combine two or more different types. For example, a **Star of Stars** network.